SDK User Reference Manual for ATMx2xx-x1x

Revision History

Date	Version	Description
March 30, 2021	0.50	Preliminary version created.
September 29, 2021	0.51	Updated 5.5 rv_prf_itf_get, 7 NVDS, 7.2 Tags, Appendix A - Known Issues.

CONFIDENTIAL

©2021, Atmosic Technologies Inc. All rights reserved. Atmosic logo is a registered trademark of Atmosic Technologies Inc. All other trademarks are the properties of their respective holders. This preliminary document is subject to change without notice.

Table of Contents

1 Purpose 6

2 ARM Cortex-M0 CPU Boot Process 6

2.1 Stack Pointer and Program Counter 6

2.2 Benign boot check 6

2.3 Set up ROM C environment 6

2.4 ROM main() 7

2.5 Boot watchdog 7

2.6 One-Time Programmable (OTP) driver initialization 7

2.7 Bind Non-Volatile Data Storage (NVDS) to OTP 7

2.8 LED driver initialization 7

2.9 Read device calibration data 8

2.10 Wake up from power saving modes 8

2.11 Calibrate hardware modules 8

2.12 Check for user application in OTP NVDS 8

2.13 Check for flash and valid application or image 9

2.14 Initialize Bluetooth LE interrupt hardware and software stack 11

2.15 Configure power management 11

2.16 Apply any register workarounds 11

2.17 Normal watchdog 12

3 Event Loop and Power Savings 12

3.1 Bluetooth LE scheduler 12

3.2 Platform scheduler 12

3.3 Poke the watchdog 12

3.4 Power management 12

3.5 PARAM_ID_SLEEP_ENABLE 13

3.6 Platform sleep duration 15

4 Interrupt 16

4.1 ROM vectors 16

4.2 HW interrupt sources and routing 17

4.3 ROM interrupt priorities 18

CONFIDENTIAL

5 Replacement Vectors 19
5.1 rv_plf_reset 19
5.2 rv_appm_init 19
5.3 rv_appm_set_adv_data 19
5.4 rv_plf_schedule 19
5.5 rv_prf_itf_get 20
5.6 rv_secure_rand_word 20
5.7 rv_prevent_ble_sleep 20
5.8 rv_plf_prevent_retention 20
5.9 rv_plf_prevent_hibernation 20
5.10 rv_plf_awoken 20
5.11 rv_plf_to_deep_sleep, rv_plf_from_deep_sleep 21
5.12 rv_plf_hibernate, rv_plf_retain_all, rv_plf_back_from_retain_all 21

6 Flash Upgrade 21
6.1 Flash partitioning 21
6.2 Flash partition header 21
6.3 First boot of newly programmed partition 23
6.4 Boot partition selection 24

7 NVDS 26
7.1 Flash File System 26
7.2 Tags 26

8 ROM Buffers and Capacities 44
8.1 Bluetooth LE heaps 44
8.2 Bluetooth LE Exchange Memory 44

Appendix A - Known Issues 45

NVDS Tags **27**
PARAM_ID_BD_ADDRESS
PARAM_ID_DEVICE_NAME
NVDS_TAG_LTK_2
NVDS_TAG_LTK_3
NVDS_TAG_APP_BLE_ACT_STRT_CMD
NVDS_TAG_APP_BLE_ACT_CRT_CMD
PARAM_ID_LPCLK_DRIFT
NVDS_TAG_APP_BLE_RSTRT_DUR

CONFIDENTIAL

PARAM_ID_MAX_SLEEP_DUR
NVDS_TAG_APP_BLE_ADV_DATA
NVDS_TAG_APP_BLE_SCAN_RESP_DATA
PARAM_ID_EXT_WAKEUP_TIME
PARAM_ID_OSC_WAKEUP_TIME
PARAM_ID_RM_WAKEUP_TIME
PARAM_ID_SLEEP_ENABLE
PARAM_ID_EXT_WAKEUP_ENABLE
PARAM_ID_ACTIVITY_MOVE_CONFIG
PARAM_ID_SCAN_EXT_ADV
PARAM_ID_SCHED_SCAN_DUR
NVDS_TAG_APP_WURX_BLE_ACT_STRT_CMD
NVDS_TAG_APP_WURX_BLE_ACT_CRT_CMD
NVDS_TAG_APP_WURX_BLE_ADV_DATA
NVDS_TAG_APP_WURX_BLE_SCAN_RESP_DATA
PARAM_ID_SLEEP_ADJ
PARAM_ID_SLEEP_ALGO_DUR
PARAM_ID_DIAG_BLE_HW
PARAM_ID_DIAG_SW
NVDS_TAG_RECENT_BOND
NVDS_TAG_MOUSE_SAMPLE_RATE
NVDS_TAG_PERIPH_BONDED
NVDS_TAG_MOUSE_NTF_CFG
NVDS_TAG_MOUSE_TIMEOUT
NVDS_TAG_PEER_BD_ADDRESS
NVDS_TAG_MOUSE_ENERGY_SAFE
NVDS_TAG_LTK
NVDS_TAG_PERIPH_BONDED_2
NVDS_TAG_PERIPH_BONDED_3
NVDS_TAG_PEER_BD_ADDRESS_2
NVDS_TAG_PEER_BD_ADDRESS_3
PARAM_ID_LE_PRIVATE_KEY_P256
PARAM_ID_LE_PUBLIC_KEY_P256
PARAM_ID_LE_DBG_FIXED_P256_KEY
PARAM_ID_LE_CODED_PHY_500
NVDS_TAG_LOC_IRK
NVDS_TAG_PEER_IRK
NVDS_TAG_PEER_IRK_2
NVDS_TAG_ADV_DURATION
NVDS_TAG_SCAN_SETTING
NVDS_TAG_WURX_PROFILE
NVDS_TAG_PEER_IRK_3
SYDNEY_TAG_RIF_CAL
SYDNEY_TAG_MDM_CAL

CONFIDENTIAL

SYDNEY_TAG_BLE_RADIO_OVR
SYDNEY_TAG_FLASH_POWERUP_DELAY
SYDNEY_TAG_PMU_WURX
SYDNEY_TAG_WATCHDOG
SYDNEY_TAG_USER_APP
SYDNEY_TAG_PMU_WKUP
SYDNEY_TAG_ATE
SYDNEY_TAG_HIBERNATE_ADJ
SYDNEY_TAG_RETAIN_ADJ
SYDNEY_TAG_SLEEP_MIN
SYDNEY_TAG_MISC_CAL
SYDNEY_TAG_CUST_CFG
SYDNEY_TAB_RADIO_CAL
SYDNEY_TAG_LED_ENABLE
SYDNEY_TAG_PMU_W
SYDNEY_TAG_MEM_W
SYDNEY_TAG_MEM_RMW
NVDS_TAG_APP Tags

List of Tables

Table 1 - Memory Reservations for User Applications
Table 2 - Memory Reservations for Third-party OS Images
Table 3 - Global Variables after Bluetooth LE Initialization
Table 4 - Interrupt Routing Configuration, Interrupt enum and RAM Vector Table Relationship

CONFIDENTIAL

1 Purpose

This document describes the operation of the ATMx2xx-x1x ROM. It also explains how user-developed code can cooperate with the ROM to implement custom Bluetooth LE applications that support advanced power management.

For a complete Doxygen-style reference of all APIs provided by this SDK, please visit customer support portal.

For documentation on ROM Bluetooth LE functionality, including GAP, GATT, and various profiles, please see the doc/ble directory.

For a complete reference of all hardware registers, refer to Register Specification for ATMx2xx-x1x.

2 ARM Cortex-M0 CPU Boot Process

This section describes the boot process of the ARM Cortex-M0 CPU. This boot can happen for a number of reasons including: power-on, external pin reset, software reset, watchdog reset, leaving hibernation, etc.

2.1 Stack Pointer and Program Counter

Immediately after reset, the ARM Cortex-M0 vector table maps directly to the beginning of the ROM device. First, the Stack Pointer (SP) is loaded from the ROM address 0x00000000. For this version (ATMx2xx-x1x), the value is 0x20014000. The CPU also loads the Program Counter (PC) from the reset vector at ROM address 0x00000004. That value is 0x00005531, which is the beginning of the reset handler, also located in the ROM device. For more information, see the provided Cortex-M0 User Guide, sections 2.1.3 and 2.3.4.

2.2 Benign boot check

The first 5 instructions of the ROM's reset handler provide a mechanism to guarantee that the Debug Access Port (DAP) can gain control of the ARM Cortex-M0's AHB-Lite bus after a reset. When P32 (aka: GPIO1 pin 12) is asserted, this code will execute a WFI instruction. This will idle the CPU and, until it is reset, keep it off the AHB-Lite bus. The evaluation board (EVB) design includes a weak pull-down on P32 to allow the CPU to continue the boot process.

2.3 Set up ROM C environment

The majority of the ROM was developed in C using Newlib for support. Before handing control to Newlib, the CMSDK_AT_DMA module is used to quickly and efficiently initialize the ROM's data section in RAM from its copy in the ROM. Then a branch is executed to Newlib's _start entry point. Newlib will perform a number of initializations including:

1) Zeroing out the ROM's .bss section

CONFIDENTIAL

2) Invoking ROM constructors
3) Invoking the ROM's main()

2.4 ROM main()

The ROM is now squarely in an execution environment for C code, and it will never return from its own main().

2.5 Boot watchdog

The boot, up to this point, has been extremely deterministic and robust. From this point on, however, user configuration and external devices will come into play that can disrupt the process. A boot watchdog is used to monitor this more dynamic portion of the boot. Such a mechanism is instrumental in providing a good user experience for automatic recovery of failed software upgrades.

The watchdog hardware module is enabled and loaded with a timeout value of 0x0a000000. With a 16 MHz system clock, this is a little over 10 seconds. This duration is extremely generous compared to the few milliseconds that are typically needed to transition from booting to operating.

Note that third-party operating systems and user applications designed not to return to the ROM's event loop will need to respect this boot watchdog in order to keep from being reset. See more below re: handing off to OTP and flash-based applications.

2.6 One-Time Programmable (OTP) driver initialization

The EFUSE_AUTOREAD word is read and retained for future reference. When NO_32KHZ_XTAL and IGNORE_32KHZ_XTAL_CHECK are not set, a handler is installed to prevent Bluetooth LE sleep until the 32 KHz xtal has settled (approximately 500 ms). This is done to preserve the accuracy of Bluetooth LE sleep clock correction.

2.7 Bind Non-Volatile Data Storage (NVDS) to OTP

This step allows NVDS methods to operate on data stored in the first 4092 bytes of OTP. It provides access to manufacturing calibration data for all types of applications and ultimately to customer configuration for designs without flash.

2.8 LED driver initialization

The SYDNEY_TAG_LED_ENABLE NVDS tag can be used to individually enable
status outputs for internal ROM operations. GPIO0 bits 3 through 9 correspond to pins P3 - P9.

LED0, P3_GPIO_3_: Deep sleep
LED1, P4_GPIO_4_: LED0 ? Hibernation : CPU WFI (when LED0 is on, LED1 indicates Hibernation. When LED0 is off, LED1 indicates CPU WFI)
LED2, P5_GPIO_5_: Bluetooth LE sleeping

CONFIDENTIAL

LED3, P6_GPIO_6_: Retention
LED4, P7_GPIO_7_: ECC computation
LED5, P8_GPIO_8_: Active tick
LED6, P9_GPIO_9_: Bluetooth LE waking up

2.9 Read device calibration data

The SYDNEY_TAG_MISC_CAL, SYDNEY_TAG_CUST_CFG, and SYDNEY_TAG_RADIO_CAL NVDS tags are read from OTP and retained in RAM. They can be accessed by user applications via the following symbols:

```
extern struct misc_cal_s misc_cal;
extern nvds_tag_len_t misc_cal_len;
extern struct cust_cfg_s cust_cfg;
extern nvds_tag_len_t cust_cfg_len;
extern struct radio_cal_s radio_cal;
extern nvds_tag_len_t radio_cal_len;
```

2.10 Wake up from power saving modes

The ROM will now begin to differentiate between a cold boot (all digital logic reset) and a warm boot (always-on digital domain preserved).

For a warm boot (ex: coming out of hibernation), the BLE hardware module may be powered off and needs to be powered on before it can be initialized below. In addition, the flash device may be in a deep power-down state and needs a wakeup before being accessed below. Pin states latched during hibernation will revert to their power-on-reset state after warm boot.

Certain flash devices (ex: Macronix) require a minimum amount of time to pass from power on to first access. To support this, when SYDNEY_TAG_FLASH_POWERUP_DELAY is present in OTP, the ROM will delay until CMSDK_PSEQ->CURRENT_REAL_TIME has passed the configured value.

2.11 Calibrate hardware modules

During a cold boot, the ROM will now calibrate a variety of hardware modules from the previously retained misc_cal, cust_cfg, and radio_cal as well as from NVDS tags SYDNEY_TAG_PMU_WKUP, SYDNEY_TAG_PMU_WURX, SYDNEY_TAG_RIF_CAL, and SYDNEY_TAG_MDM_CAL. It will also power off the unused second bank of EMRAM.

2.12 Check for user application in OTP NVDS

If the ROM finds the SYDNEY_TAG_USER_APP NVDS tag in OTP, it will load it to the RAM address 0x20014000 and jump to it.

CONFIDENTIAL

Here is a summary of the environment created by the ROM when handing off to an OTP user application:

- Stack pointer will point to valid RAM region
- ROM will have initialized its own sections in RAM
- Boot watchdog will be enabled and running (approx 10 seconds)
- ROM NVDS methods will be bound to OTP data
- misc_cal, cust_cfg, and radio_cal will be populated
- Bluetooth LE hardware module will be powered on
- External flash will be awake or waking up
- Hardware modules will be calibrated
- Second bank of EMRAM will be powered off

2.13 Check for flash and valid application or image

Based on the package settings in the EFUSE_AUTOREAD word, the ROM will configure the pinmux to enable a QSPI interface. Please consult the SDK pinmux tool for the specific settings.

The ROM will enable the doubled 32 MHz "hyper" flash clock and begin flash discovery (via the SPI2 module) looking for supported Macronix, Micron, GigaDevice and Winbond parts. If a supported part is found and can be configured for the needed high-performance access mode, the QSPI AHB bridge will be configured to memory map the device starting at address 0x10000000.

Next, the ROM will read from the memory mapped flash device and program QSPI_REMOTE_AHB_SETUP_4__INVERT_ADDR to remap the active bank as needed.
See Flash Upgrade section for full details.

Finally, the ROM will read from the active bank and look for the signature of either a user application or a third-party OS image.

Here is a summary of the environment created by the ROM when handing off to a flash-based user application:

- Stack pointer will point to valid RAM region (less than 0x20014000)
- ROM will have initialized its own sections in RAM
- Boot watchdog will be enabled and running (approx 10 seconds)
- INTRPT_CFG_15 (DMA_IRQn) will be configured for only INTISR_SRC_BLE
- DMA_IRQn will be enabled with IRQ_PRI_NORMAL priority
- CMSDK_AT_DMA->INTERRUPT_MASK will be disabled
- ROM NVDS methods will be bound to OTP data
- misc_cal, cust_cfg, and radio_cal will be populated
- Bluetooth LE hardware module will be powered on

CONFIDENTIAL

- Hardware modules will be calibrated
- Second bank of EMRAM will be powered off
- User application in flash will be mapped to address 0x10000000
- ROM will jump to the first word of the user application

Table 1 - Memory Reservations for User Applications

Address Range	Allocation
0x00000000-0x00040000	ROM itself
0x10000000-0x10ffffff	QSPI flash (user application)
0x20000000-0x20000ac3	RAM - ROM's .data section
0x20000ac4-0x20001973	RAM - ROM's .bss section
0x20001974-0x20007dbf	RAM - ROM's reserved Bluetooth LE heap
0x20007dc0->	RAM - heap for ROM and user applications
<-0x20013fff	RAM - stack for ROM and user applications
0x20014000-0x20017fff	RAM - Region for user applications
0x30008000-0x3000b981	EMRAM - Bluetooth LE data
0x3000b982-0x3000bfff	EMRAM - unused
0x3000c000-0x3000ffff	EMRAM - unused and powered off

CONFIDENTIAL

Here is a summary of the environment created by the ROM when handing off to a flash-based OS image:

- Stack pointer will be loaded from first word of OS image (ex: 0x20018000)
- Entire RAM will essentially be uninitialized
- Boot watchdog will be enabled and running (approx 10 seconds)
- Bluetooth LE hardware module will be powered on
- Hardware modules will be calibrated
- Second bank of EMRAM will be powered off
- OS image in flash will be mapped to address 0x00000000 (hiding ROM)
- ROM will jump to the OS image reset vector (from second word of OS image)

Table 2 - Memory Reservations for Third-party OS Images

Address Range	Allocation
0x00000000-0x00ffffff	QSPI flash (OS image)
0x20000000-0x20017fff	RAM
0x30008000-0x3000bfff	EMRAM
0x3000c000-0x3000ffff	EMRAM, but powered off

If a valid application/image cannot be located, the flash device will be placed into its lowest power mode. All failure modes will leave this step with the SPI2 and QSPI modules in reset and the 32 MHz clock disabled.

2.14 Initialize Bluetooth LE interrupt hardware and software stack

ROM execution will continue here when there is no flash or when the flash does not have a valid application/image. In those cases, NVDS will still be bound to OTP data. Execution can also continue here when a flash-based user application has finished initialization and returns. Often, that will mean that NVDS is bound to data stored on the flash rather than OTP.

The ROM Bluetooth LE driver uses INTRPT_CFG_15 (aka: DMA_IRQn). The routing is set exclusively to INTISR_SRC_BLE and the interrupt is enabled with IRQ_PRI_NORMAL priority.

The Bluetooth LE hardware and full SW stack will be initialized next. During this process, the SYDNEY_TAG_BLE_RADIO_OVR NVDS tag can provide parameter overrides for the Bluetooth LE radio interface, and rv_appm_init/appm_init() will be invoked to initialize any Bluetooth LE application.

2.15 Configure power management

The NVDS tags SYDNEY_TAG_HIBERNATE_ADJ, SYDNEY_TAG_RETAIN_ADJ, and SYDNEY_TAG_SLEEP_MIN permit fine timing adjustments to optimize power consumption. This part of the boot will read them into memory and retain them in the following symbols:

```
extern int32_t plf_hibernate_adjust;
extern int32_t plf_retain_adjust;
extern int32_t plf_sleep_min;
```

2.16 Apply any register workarounds

The NVDS tags SYDNEY_TAG_PMU_W, SYDNEY_TAG_MEM_W, and SYDNEY_TAG_MEM_RMW support a generic catch-all mechanism for hardware and software initialization and calibration. It is recommended that these only be used when directed by Atmosic.

CONFIDENTIAL

2.17 Normal watchdog

At this point, the boot is complete. If the SYDNEY_TAG_WATCHDOG NVDS tag does not exist, the watchdog will be disabled. If it does, the watchdog hardware module will be loaded with its value and the main loop will need to periodically poke the watchdog in order to prevent the system from being reset.

3 Event Loop and Power Savings

This section explains the main event loop executed by the ROM, with particular focus on the supported power saving modes. This code is utilized in a number of situations, including:

OTP-based beacons
OTP-based user applications that return to the ROM
Flash-based user applications that return to the ROM

3.1 Bluetooth LE scheduler

The ROM's event loop begins with an invocation of the Bluetooth LE software stack's rwip_schedule(). That routine will process all active events and expired timers repeatedly until there is no work remaining.

3.2 Platform scheduler

Next, rv_plf_schedule/plf_schedule() is invoked to allow user applications and platform software to process items not managed by the Bluetooth LE software stack. This can be a handy place to poll platform devices for status or to insert periodic debug code. See driver/sensor/ICM-20948/icm.c debug_icm_schedule() for an example.

CONFIDENTIAL

3.3 Poke the watchdog

Then, the hardware watchdog module is poked by writing to its INTCLR register. This keeps it from resetting the system. For more information, see the provided Cortex-M SDK Technical Reference Manual, section 4.5.

3.4 Power management

Now that a pass has been made through the event loop, the system may be idle and ready to transition to a power saving state. While performing the items in this step, CPU interrupts are globally disabled so that any transition is atomic with respect to the system state.

The state of the Bluetooth LE software stack (including Bluetooth LE applications) is checked to make sure that power management has been enabled (see "PARAM_ID_SLEEP_ENABLE" section below) and that no active work is ongoing. Active work can include low level items such as simple events or expired timers, and mid level items such as those described by the prevent_sleep enum or that are managed via rv_prevent_ble_sleep.

The check also computes a sleep duration based upon known future events and any configured limitations:
If NVDS tag PARAM_ID_EXT_WAKEUP_ENABLE is unset or set to 0, the maximum sleep duration is:

```
#define MAX_SLEEP_DURATION_PERIODIC_WAKEUP      0x0640  // 0.5s
```

If NVDS tag PARAM_ID_EXT_WAKEUP_ENABLE is set to 1, the maximum sleep duration is:

```
#define MAX_SLEEP_DURATION_EXTERNAL_WAKEUP      0x2f000  // 1min
```

The chosen maximum sleep duration can be overridden by NVDS tag PARAM_ID_MAX_SLEEP_DUR. NVDS tag PARAM_ID_SLEEP_ADJ will be subtracted from the sleep duration. This can be used to provide (or remove) leeway between sleep and the next event.

If NVDS tag PARAM_ID_EXT_WAKEUP_ENABLE is set to 2, when the computed sleep duration is equal to the max, the computed sleep duration is set to 0, which means forever. The application layer may choose to adjust this to another duration via rv_plf_prevent_retention or rv_plf_prevent_hibernation.

If any active items are found, or the sleep duration is too short, this step is exited and the event loop will repeat. If not, the Bluetooth LE hardware module is placed in the deep sleep state and checks will be made to see if additional layers of power savings can be applied.

3.5 PARAM_ID_SLEEP_ENABLE

The NVDS tag PARAM_ID_SLEEP_ENABLE is the top level control for power management. It will be read into the sleep_enable global variable during Bluetooth LE initialization. The values are as follows:

```
#define SLEEP_ENABLE_NONE       0
```

Mode NONE means that neither Bluetooth LE nor PSEQ-based power saving is enabled.

When the system is idle, the CPU will wait inside WFI. LED1 will be on during this wait.

```
#define SLEEP_ENABLE_DEEP       1
```

Mode DEEP means that the Bluetooth LE can enter deep sleep. In other words, gating off its primary 16 MHz clock when idle. A small "always-on" domain using the 32 KHz clock will wake the Bluetooth LE and correct its counters before the next event.

CONFIDENTIAL

Once the Bluetooth LE is in deep sleep, LED0 will be turned on and rv_plf_to_deep_sleep (defaulting to plf_to_deep_sleep()) will be invoked. The CPU will wait inside WFI. Upon waking up, rv_plf_from_deep_sleep (defaulting to plf_from_deep_sleep()) will be invoked and LED0 will be turned off.

```
#define SLEEP_ENABLE_RETAIN             2
```

Mode RETAIN builds on top of DEEP. Additional system resources will be idled or shut down, while still retaining enough system state to quickly continue. For example, this can be used between radio Rx and Tx events while still maintaining a Bluetooth LE connection.

plf_to_deep_sleep() will check sleep_enable and the computed sleep duration (see "Platform sleep duration" section below). rv_plf_prevent_retention will be invoked to provide high level applications with a chance to adjust or avoid this mode. If any of these conditions are not met, plf_to_deep_sleep() will return and fall back to SLEEP_ENABLE_DEEP.

Finally, rv_plf_retain_all (defaulting to plf_retain_all()) will be invoked to enter the retain power save mode. This routine will program the wakeup timer & mask, turn on LED3, and initiate the power save mode. The primary 16 MHz clock will be shut down for the entire chip, some stateless modules will be powered off, and RAM will be placed in a state retention mode.

The ROM event loop is designed such that the CPU stack does not need to be retained, which saves power for an entire bank of RAM.

Waking up from the retain power save mode, plf_from_deep_sleep() will invoke rv_plf_back_from_retain_all (defaulting to plf_back_from_retain_all()). plf_back_from_retain_all() will turn off LED3, save the wakeup reason into the wake_status global variable, and set the KE_EVENT_WAKE_STATUS event.

```
#define SLEEP_ENABLE_RETAIN_DROP 3
```

Mode RETAIN_DROP builds on top of RETAIN by also lowering the voltage of certain internal power rails. Note that this voltage lowering will not take place when the wake-up receiver (WURX) is enabled.

```
#define SLEEP_ENABLE_HIBERNATE          4
```

Mode HIBERNATE builds on top of RETAIN_DROP, but in a different manner. All of the previous modes will preserve system state, but hibernation will cause most of the system state to be lost. Therefore, it is essential for hibernation to interact with high level application state, such as whether there is an active Bluetooth LE connection or not.

plf_to_deep_sleep() will check sleep_enable and the computed sleep duration (see Platform sleep duration section below). rv_plf_prevent_hibernation will be invoked to provide high level applications

CONFIDENTIAL

with a chance to adjust or avoid this mode. If any of these conditions are not met, plf_to_deep_sleep() will fall back to SLEEP_ENABLE_RETAIN_DROP.

Finally, rv_plf_hibernate (defaulting to plf_hibernate()) will be invoked to enter the hibernate power save mode. This routine will program the wakeup timer & mask, turn on LED1, and initiate the power save mode. All digital circuits will be shut down except for the "always-on" domain.

Waking up from the hibernate power save mode is similar to a power-on reset. The reset of digital circuits will automatically turn off LED1. The CPU will boot via the ROM.

```
#define SLEEP_ENABLE_SOC_OFF            5
```

Mode SOC_OFF is a deeper, lower power form of HIBERNATE with limited wakeup capabilities.

plf_hibernate() will check the sleep enable, the computed sleep duration, and the wakeup mask for compatibility. If any of these conditions are not met, it will fall back to SLEEP_ENABLE_HIBERNATE.

plf_soc_off() will be invoked to program the wakeup timer on top of the configuration from NVDS tag SYDNEY_TAG_PMU_WKUP. All digital circuits will be shut down including the "always-on" domain. All pins under pinmux control will revert back to the reset state.

Waking up from the soc_off power save mode is almost exactly the same as a power-on reset.

3.6 Platform sleep duration

The following NVDS tags are read into global variables after Bluetooth LE initialization:

Table 3 - Global Variables after Bluetooth LE Initialization

NVDS Tag	Global Variable
SYDNEY_TAG_HIBERNATE_ADJ	plf_hibernate_adjust
SYDNEY_TAG_RETAIN_ADJ	plf_retain_adjust
SYDNEY_TAG_SLEEP_MIN	plf_sleep_min

When the above logic decides to enter HIBERNATE or SOC_OFF, plf_hibernate_adjust is subtracted from the computed Bluetooth LE sleep duration to determine the power save duration.

When the above logic decides to enter RETAIN or RETAIN_DROP, plf_retain_adjust is subtracted from the computed sleep duration to determine the power save duration.

The plf_sleep_min variable represents the minimum duration for entering any power save mode.

CONFIDENTIAL

4 Interrupt

This section provides interrupt details for applications that work in conjunction with the ROM. It does not apply to third-party OS images, as they provide their own vector table.

4.1 ROM vectors

The CPU vector table (see Cortex-M0 User Guide, section 2.3.4) maps to a section of the ROM device. Code in said ROM device dispatches to a secondary vector table that lives in RAM and that is initialized by the ROM startup code. This allows applications to install their own interrupt handlers.

Take the NMI vector as an example:

```
platform/atm2/ATM2xxx-x1x% ../../../tools/gcc-arm-none-eabi-8-2019-q3-
update/bin/arm-none-eabi-gdb rom/fw.elf
(gdb) x/x 0x8
0x8 <__isr_vector+8>: 0x0000556d
(gdb) disassemble 0x0000556d
Dump of assembler code for function __rom__NMI_Handler:
0x0000556c <+0>: ldr r3, [pc, #4] ; (0x5574 <__rom__NMI_Handler+8>)
0x0000556e <+2>: ldr r2, [r3, #0]
0x00005570 <+4>: bx r2
...
(gdb) x/x 0x5574
0x5574 <__rom__NMI_Handler+8>: 0x20000000
(gdb) x/x 0x20000000
0x20000000 <__ram_isr_vector__NMI_Handler>: 0x00007b09
(gdb) disassemble 0x00007b09
Dump of assembler code for function NMI_Handler:
0x00007b08 <+0>: push {r4, lr}
0x00007b0a <+2>: ldr r3, [pc, #36] ; (0x7b30 <NMI_Handler+40>)
0x00007b0c <+4>: ldr r2, [r3, #0]
0x00007b0e <+6>: adds r1, r2, #1
…
```

Notice that __ram_isr_vector__NMI_Handler lives in RAM and points to a default NMI_Handler in the ROM. There are 37 such __ram_isr_vector__* symbols, and they are defined in vectors.h.

CONFIDENTIAL

4.2 HW interrupt sources and routing

Table 4 shows the relationship between the CMSDK_WRPR interrupt routing configuration, the IRQn enum (in CMSDK_CM0.h), and the RAM vector table (in vectors.h):

Table 4 - Interrupt Routing Configuration, Interrupt enum and RAM Vector Table Relationship

CMSDK_WRPR	IRQn enum in CMSDK_CM0.h	RAM vector table in vectors.h
INTRPT_CFG_0	UARTRX0_IRQn	__ram_isr_vector__UARTRX0_Handler
INTRPT_CFG_1	UARTTX0_IRQn	__ram_isr_vector__UARTTX1_Handler
INTRPT_CFG_2	UARTRX1_IRQn	__ram_isr_vector__UARTRX1_Handler
INTRPT_CFG_3	UARTTX1_IRQn	__ram_isr_vector__UARTTX1_Handler
INTRPT_CFG_4	UARTRX2_IRQn	__ram_isr_vector__UARTRX2_Handler
INTRPT_CFG_5	UARTTX2_IRQn	__ram_isr_vector__UARTTX2_Handler
INTRPT_CFG_6	PORT0_ALL_IRQn	__ram_isr_vector__PORT0_COMB_Handler
INTRPT_CFG_7	PORT1_ALL_IRQn	__ram_isr_vector__PORT1_COMB_Handler
INTRPT_CFG_8	TIMER0_IRQn	__ram_isr_vector__TIMER0_Handler
INTRPT_CFG_9	TIMER1_IRQn	__ram_isr_vector__TIMER1_Handler
INTRPT_CFG_10	DUALTIMER_IRQn	__ram_isr_vector__DUALTIMER_HANDLER
INTRPT_CFG_11	SPI_IRQn	__ram_isr_vector__SPI_Handler
INTRPT_CFG_12	UARTOVF0_IRQn	__ram_isr_vector__UARTOVF0_Handler
INTRPT_CFG_13	UARTOVF1_IRQn	__ram_isr_vector__UARTOVF1_Handler
INTRPT_CFG_14	UARTOVF2_IRQn	__ram_isr_vector__UARTOVF2_Handler
INTRPT_CFG_15	DMA_IRQn	__ram_isr_vector__DMA_Handler

The intisr.h file describes 32 interrupt sources as a bitmask. Each of the above 16 interrupt destinations can be configured to respond to any number of the 32 sources.

CONFIDENTIAL

For example, consider the uart0 driver:

```
    // Connect interrupt source to destination
    CMSDK_WRPR->INTRPT_CFG_0 = INTISR_SRC_UART0_RXINT;
    CMSDK_WRPR->INTRPT_CFG_1 = INTISR_SRC_UART0_TXINT;
    CMSDK_WRPR->INTRPT_CFG_12 = INTISR_SRC_UART0_OVERFLOW;
    // Clear pending interrupt in NVIC
    NVIC_ClearPendingIRQ(UARTRX0_IRQn);
    NVIC_ClearPendingIRQ(UARTTX0_IRQn);
    NVIC_ClearPendingIRQ(UARTOVF0_IRQn);
    // Install interrupt handler
    __ram_isr_vector__UARTRX0_Handler = UARTRX0_Handler;
    __ram_isr_vector__UARTTX0_Handler = UARTTX0_Handler;
    __ram_isr_vector__UARTOVF0_Handler = UARTOVF0_Handler;



    CMSDK_WRPR->INTRPT_CFG_15 = INTISR_SRC_BLE;
```

4.3 ROM interrupt priorities

The interrupt routing feature provides significant control with respect to interrupt source combining. Because interrupt sources are not hard-wired together, it is possible to make effective use of interrupt priorities across the system.

The following macros in ll.h suggest a strategy for interrupt priorities:

```
#define IRQ_PRI_RT  0     // Real time - highest priority
#define IRQ_PRI_HIGH    1
#define IRQ_PRI_NORMAL    2
#define IRQ_PRI_UI  3     // User interface - lowest priority
```

For example, the UI priority is used by human user interface drivers such as the "ps2" mouse:

```
    CMSDK_WRPR->INTRPT_CFG_7 = INTISR_SRC_GPIO1_COMB;
    CMSDK_WRPR->INTRPT_CFG_8 = INTISR_SRC_TIMER0;
    NVIC_SetPriority(PORT1_ALL_IRQn, IRQ_PRI_UI);
    NVIC_SetPriority(TIMER0_IRQn, IRQ_PRI_UI);
```

While initializing Bluetooth LE, the ROM configures the following interrupt priority:

```
    NVIC_SetPriority(DMA_IRQn, IRQ_PRI_NORMAL);
```

CONFIDENTIAL

5 Replacement Vectors

5.1 rv_plf_reset

When a system reset is requested by the ROM software, this vector will be invoked after interrupts are globally disabled but before asserting the hardware reset. It is the absolute last opportunity for drivers to take action before all system state is lost.

rv_plf_reset has no default handler.

5.2 rv_appm_init

At the end of Bluetooth LE device initialization, this vector will be invoked to initialize the Bluetooth LE application layer. It provides the final opportunity for customization before the main event loop is entered.
Flash-based user applications should use this vector to initialize their own custom Bluetooth LE applications. See BLE_adv, BLE_scan, HCI, HID_keyboard, and HID_mouse for examples.

The default handler for rv_appm_init is the ROM's built-in Bluetooth LE beacon application, which is configurable via OTP NVDS tags.

5.3 rv_appm_set_adv_data

This vector can be used to customize the GAPM_SET_ADV_DATA_CMD performed by the ROM's built-in beacon application. It is particularly useful for tiny OTP user applications to place sensor data into the advertisement.

CONFIDENTIAL

5.4 rv_plf_schedule

After all active events have been processed by the main event loop, this vector is invoked before interrupts are disabled and power saving modes are handled.

The default handler for rv_plf_schedule reads a tick counter from PSEQ and sets LED 5 based upon the status of bit 14:

```
static void plf_schedule(void)
{
    // Fire up PSEQ
    CMSDK_WRPR1->APB8_CTRL = WRPR_APB8_CTRL__CLK_ENABLE__MASK;
    uint32_t tick = CMSDK_PSEQ->CURRENT_REAL_TIME;
    // Done with PSEQ
    CMSDK_WRPR1->APB8_CTRL = 0;

    (tick & (1 << 14)) ? led_set(5) : led_reset(5);
}
```

5.5 rv_prf_itf_get

This vector supports the addition and replacement of Bluetooth LE profiles. See profiles driver for more information.

The default handler dispatches to Bluetooth LE profiles that are built into the ROM.

5.6 rv_secure_rand_word

This vector can be used to provide a secure source of random numbers to various Bluetooth LE security functions, such as key generation. See platform/atm2/ATM2xE3c/driver/trng for more information.

The default handler for rv_secure_rand_word is the ROM's copy of rand(), which is not secure.

5.7 rv_prevent_ble_sleep

This vector can be used to prevent the Bluetooth LE module from entering sleep. It will also prevent the rest of the system from entering any power saving mode.

rv_prevent_ble_sleep has no default handler, but the ROM does install a handler in the chain to keep the Bluetooth LE module awake when the 32 KHz xtal is present but not yet ready to be a stable time reference.

5.8 rv_plf_prevent_retention

After the Bluetooth LE module has gone to sleep, this vector can be used to prevent the system from entering the retention power saving mode.

rv_plf_prevent_retention has no default handler.

5.9 rv_plf_prevent_hibernation

After the Bluetooth LE module has gone to sleep, this vector can be used to prevent the system from entering the hibernation power saving mode. The HID examples use this to protect the system state while there is an active connection to a central host.

rv_plf_prevent_hibernation has no default handler, but the ROM's beacon application will install a handler in the chain to prevent hibernation while there is active GAPM activity.

5.10 rv_plf_awoken

After the system wakes up from a power saving mode, this vector is invoked from a ke_event handler with the wake-up reason from the PSEQ STATUS register. Applications can decode this reason and take specific action.

The default handler for rv_plf_awoken notifies the ROM's beacon application when the system is woken up due to a WURX event.

CONFIDENTIAL

5.11 rv_plf_to_deep_sleep, rv_plf_from_deep_sleep

This vector permits customization of the ROM's power saving logic.
See user/user_loop.c for more information.

5.12 rv_plf_hibernate, rv_plf_retain_all, rv_plf_back_from_retain_all

This vector permits customization of the ROM's power saving logic.
See user/user_loop.c for more information.

6 Flash Upgrade

This section describes the hardware facilities and ROM code that support in-service upgrades of flash installations. It also explains the complete process for performing such an upgrade.

6.1 Flash partitioning

The ROM has the ability to divide a flash device in half (one active and one for upgrade) and conditionally swap the two halfs within its memory map. This allows a flash application or OS image to always run from the same memory address independent of which half of the flash it was actually written to.

For example, consider a 4 Mb flash (total length 0x80000 bytes). An application linked for memory range 0x10000000-0x1003ffff or an OS image linked for memory range 0x00000000-0x0003ffff can live at offset 0x00000 or offset 0x40000 inside the flash. Conveniently, once the active half has been mapped, memory range 0x10040000-0x1007ffff will correspond to the other half of the flash where an upgrade can take place.

Here is a snippet of the ROM's map_b1() routine that swaps the two halfs in memory:

```
DEBUG_TRACE("Swapping in flash bank 1");
CMSDK_QSPI->REMOTE_AHB_SETUP_4 =
   QSPI_REMOTE_AHB_SETUP_4__INVERT_ADDR__WRITE(1 << (mem_cap - 1));
```

6.2 Flash partition header

In order to decide which half (henceforth referred to as partition) of the flash should be used during the ROM boot process, a special word in each of the partitions is consulted:

```
#define FLASH_UPGRADE_STATUS_WOFF       4
```

When the flash partition contains a third-party OS image, this is a reserved word in the Cortex-M0 vector table and corresponds to exception number 4.

CONFIDENTIAL

When the flash partition contains a flash application, the first 4 words of startup code must eventually jump over this location.

An example from user_startup_CMSDK_CM0.s:

```
_user_entry:
    ldr    r0, =0x70000008      /* CMSDK_AT_DMA->SRC_ADDR */
    ldr    r1, =__etext
    ldr    r2, =__data_start__
    ldr    r3, =__data_end__
    subs   r3, r2
    stm    r0!, {r1-r3}         /* SRC_ADDR, TAR_ADDR, SIZE */
    subs   r0, #0x14            /* CMSDK_AT_DMA */
    b      _upgrade_data_end
_upgrade_data:
    .byte  0xfe                 /* Not erased */
    .ascii "USR"
    .long  0xffffffff
    .long  0xffffffff
    .long  0xffffffff
_upgrade_data_end:
    movs   r1, #0x80
    ...
```

The first byte (note: little-endian) contains the partition status:

```
#define FLASH_UPGRADE_STATUS_MASK             0x000000ff
```

The lower nibble of this byte is a state bitmask:

```
#define FLASH_UPGRADE_STATUS_STATE_MASK       0x0f
```

The breakdown of those 4 bits is as follows:

```
#define FLASH_UPGRADE_STATUS_ERASED_MASK      0x01
```

When this bit is set, it indicates that the flash partition is erased. This will happen automatically during a standard flash erase. It is recommended that this bit be cleared in the binary image before programming.

```
#define FLASH_UPGRADE_STATUS_nPROGRAMMED_MASK 0x02
```

When this bit is clear, it indicates that the flash partition has been successfully programmed. It is recommended that this bit be set in the binary image before programming, and then cleared by the install mechanism after the entire image has been programmed.

```
#define FLASH_UPGRADE_STATUS_nBOOTED_MASK     0x04
```

When this bit is clear, it indicates that the flash partition has been booted and is known good. It is recommended that this bit be set in the binary image before programming, and then cleared by the application itself after it is completely up and running.

CONFIDENTIAL

```
#define FLASH_UPGRADE_STATUS_nBAD_MASK          0x08
```

When this bit is clear, it indicates that the flash partition is known to be bad. It is recommended that this bit be set in the binary image before programming, and any part of the upgrade process may clear it to indicate failure.

The upper nibble is a sequence number:

```
#define FLASH_UPGRADE_STATUS_SEQ_MASK           0xf0
```

It is recommended that all of these bits be set in the binary image before programming, and then programmed to the final value by the application itself after it is completely up and running.

The next 3 bytes provide an ASCII identifier of the partition's content.

```
#define FLASH_UPGRADE_STATUS_ID_MASK            0xffffff00
#define FLASH_UPGRADE_STATUS_ID_ROM             0x4d4f5200
#define FLASH_UPGRADE_STATUS_ID_USR             0x52535500
```

It is recommended that these bytes be fixed in the binary image before programming.

6.3 First boot of newly programmed partition

Once a new image has been programmed into the empty partition and the system is ready to reboot, a special value is written to the CMSDK_PSEQ->PERSISTENT0 register:

```
#define FLASH_PERSIST_FIRST_BOOT_MAGIC          0xf157b007
```

The SDK ext_flash driver provides ext_flash_upgrade_and_reboot() to perform the above steps.

When the ROM sees this special value in PERSISTENT0, it will choose to boot the partition that is NOT known good, which will correspond to the newly programmed partition.

The PERSISTENT0 register will be cleared by the ROM before attempting to boot said newly programmed partition. This will automatically cause fall-back recovery to the good partition in the event of any reset. See also the 2.5 Boot watchdog section.

To finish the upgrade, as mentioned above, the new application needs to decide whether the upgrade was successful or not. This decision must be made carefully, as blessing a bad image can prevent future in-service upgrades from taking place and require reprogramming via physical access to the design. Once good, the nBOOTED bit should be cleared and the SEQ should be programmed to the current known good SEQ plus one.

See ext_flash_is_first_boot() and ext_flash_mark_boot_good() in the SDK ext_flash driver.

CONFIDENTIAL

6.4 Boot partition selection

The whole selection process looks like this:

```
{
    uint32_t bank_off = (1 << (mem_cap - 3));
    uint32_t b0_us = __UserFlash[FLASH_UPGRADE_STATUS_WOFF];
    uint32_t b1_us = __UserFlash[bank_off + FLASH_UPGRADE_STATUS_WOFF];

    // Special case for image that does not support bank upgrade process
    if (b0_us == 0) {
        return;
    }

    bool first_boot = false;
    if (CMSDK_PSEQ->PERSISTENT0 == FLASH_PERSIST_FIRST_BOOT_MAGIC) {
       CMSDK_PSEQ->PERSISTENT0 = 0;
       first_boot = true;
    }

    bool b0 = id_is_valid(b0_us);
    bool b1 = id_is_valid(b1_us);

    if (b0) {
        if (first_boot && bank_is_programmed(b0_us)) {
            // Attempt first boot of b0
            DEBUG_TRACE("Flash bank 0 first boot");
            b1 = false;
            goto done;
        }
        b0 = status_is_all_good(b0_us);
    }

    if (b1) {
        if (first_boot && bank_is_programmed(b1_us)) {
            // Attempt first boot of b1
            DEBUG_TRACE("Flash bank 1 first boot");
            b0 = false;
            goto done;
        }
        b1 = status_is_all_good(b1_us);
    }

    // When in doubt, default to b0.  Evidence required to switch to b1.
done:
    if (!b1) {
        return;
    }

    if (!b0) {
        map_b1(mem_cap);
        return;
    }
```

CONFIDENTIAL

```
    map_winning_seq((b0_us & FLASH_UPGRADE_STATUS_SEQ_MASK) >> 4,
                    (b1_us & FLASH_UPGRADE_STATUS_SEQ_MASK) >> 4, mem_cap);
}

A few of the support routines follow.

static bool
id_is_valid(uint32_t upgrade_status)
{
    uint32_t id = upgrade_status & FLASH_UPGRADE_STATUS_ID_MASK;
    return ((id == FLASH_UPGRADE_STATUS_ID_ROM) ||
            (id == FLASH_UPGRADE_STATUS_ID_USR));
}

// A known good partition will not be erased, will be programmed, will be
// booted, and will not be bad.
static bool
status_is_all_good(uint32_t upgrade_status)
{
    return ((upgrade_status & FLASH_UPGRADE_STATUS_STATE_MASK) ==
            FLASH_UPGRADE_STATUS_nBAD_MASK);
}

// A programmed partition will not be erased, will be programmed, will not
// be booted, and will not be bad.
static bool
bank_is_programmed(uint32_t upgrade_status)
{
    return ((upgrade_status & FLASH_UPGRADE_STATUS_STATE_MASK) ==
            (FLASH_UPGRADE_STATUS_nBAD_MASK |
             FLASH_UPGRADE_STATUS_nBOOTED_MASK));
}

// When the ROM discovers two good partitions, if the second partition
// has a sequence number equal to the first partition's sequence number,
// then the second good partition will be used.  Otherwise, the first good
// partition will be used.
static void
map_winning_seq(uint8_t seq0, uint8_t seq1, uint8_t mem_cap)
{
    if (((seq0 + 1) & 0xf) == seq1) {
        map_b1(mem_cap);
    }
}
```

CONFIDENTIAL

7 NVDS

7.1 Flash File System

NVDS (Non Volatile Data Storage) is a dictionary-like journaled file system. It is used to store configuration that can be accessed by the application and Bluetooth LE stack at run time. An example would be to specify power management related settings to control the device behavior when idle.

A datum in the dictionary can be a singleton, e.g. a beacon device address, or a compound, e.g. Bluetooth LE advertising contents. The NVDS data is keyed using NVDS tags. The tags and their description including the size and an example is listed below.

Basic operations supported by the NVDS APIs include adding, deleting, browsing or locking the tags and its data. The size of the NVDS is configurable and defaults to 32 K bytes. An internal cache keeps the access efficient through faster lookups.

It is possible for the filesystem to get full over time as the tags are added and removed. This may create holes which may prevent addition of new tags. As such, if a new tag being added is unable to find sufficient space on the filesystem, a purge operation is initiated automatically. The operation performs a read of all the valid TAGs of the NVDS, stores them in the temporary buffer allocated by the caller, erases the NVDS flash region completely and then rewrites all the valid TAGs.

The SDK also provides a tool called 'nvds_tool' which can create, modify, and view an NVDS dictionary from tag-data pairs. But it defers to the user on writing the tag data in a binary-like format called tag data source (TDS). Therefore, setting singleton NVDS data (e.g. `PARAM_ID_MAX_SLEEP_DUR`) is relatively easy and intuitive. One only needs to be aware of the sizes of various data types and byte-ordering. However, setting individual members of compound NVDS data types (e.g. `APP_BLE_ACT_CRT_CMD`) can take some effort due to the field-insensitivity of the NVDS file system -- to configure a parameter that is a member of a compound at a non-zero offset, all of the preceding members in that compound must be set also. And the user must take care of all the fields' alignment and padding.

While writing NVDS data from scratch in TDS format can be difficult, modifying existing NVDS data is easy. And the SDK already provides a variety of TDS source samples for common beacon contents and parameters. Moreover, those samples, together with the makefile targets `pull_otp_nvds`, `rebuild_otp_nvds`, and `push_otp_nvds`, are very useful in modifying NVDS data in a beacon device's OTP to change its behavior.

CONFIDENTIAL

7.2 Tags

Name	PARAM_ID_BD_ADDRESS
NVDS Tag	0x01
Size	6 bytes
Example	HID_keyboard/tag_data/01-BD_ADDRESS/keyboard_101.tds
Description	Local Board Address
Name	**PARAM_ID_DEVICE_NAME**
NVDS Tag	0x02
Size	<= 248 bytes
Example	HID_keyboard/tag_data/02-DEVICE_NAME/keyboard_101.tds
Description	Device Name
Name	**NVDS_TAG_LTK_2**
NVDS Tag	0x04
Size	sizeof(struct gapc_ltk) == 28 bytes
Description	EDIV (2 bytes), RAND NB (8 bytes), LTK (16 bytes), Key Size (1 byte) For Keyboard Slot 2
Name	**NVDS_TAG_LTK_3**
NVDS Tag	0x03
Size	sizeof(struct gapc_ltk) == 28 bytes
Description	EDIV (2 bytes), RAND NB (8 bytes), LTK (16 bytes), Key Size (1 byte) For Keyboard Slot 3
Name	**NVDS_TAG_APP_BLE_ACT_STRT_CMD**
NVDS Tag	0x05

CONFIDENTIAL

Size	sizeof(struct gapm_adv_param)
Example	OTP_beacon/tag_data/05-APP_BLE_ACT_STRT_CMD/30_adv.tds
Description	Bluetooth LE Application Activity Start CMD
Name	**NVDS_TAG_APP_BLE_ACT_CRT_CMD**
NVDS Tag	0x06
Size	sizeof(struct gapm_activity_create_adv_cmd)
Example	OTP_beacon/tag_data/06-APP_BLE_ACT_CRT_CMD/ref_legacy_nc_s_1s.tds
Description	Bluetooth LE Application Activity Create CMD
Name	**PARAM_ID_LPCLK_DRIFT**
NVDS Tag	0x07
Size	2 bytes
Default	500 (ppm)
Description	Radio Drift
Name	**NVDS_TAG_APP_BLE_RSTRT_DUR**
NVDS Tag	0x09
Size	4 bytes
Example	OTP_beacon/tag_data/09-APP_BLE_RSTRT_DUR/30s.tds
Description	Bluetooth LE Application Restart Duration (in 0.01 sec) When PARAM_ID_SLEEP_ENABLE >= Hibernate and PARAM_ID_EXT_WAKEUP_ENABLE is set to 0x02, values above 6000 will be measured in 32 KHz cycles: 0-60 seconds: 0x00000000-0x00001770 centiseconds 60+ seconds: 0x001e0000-0x7fffffff 32 KHz cycles For consistent behavior, it is advised that PARAM_ID_EXT_WAKEUP_ENABLE be set to 0x01 for restart durations below 60 seconds.
Name	**PARAM_ID_MAX_SLEEP_DUR**

CONFIDENTIAL

NVDS Tag	0x0a
Size	4 bytes
Default	PARAM_ID_EXT_WAKEUP_ENABLE ? 0x2f000 (1 min) : 0x640 (0.5 sec)
Description	Maximum sleep duration (in half slots which are 312.5 µs)
Name	**NVDS_TAG_APP_BLE_ADV_DATA**
NVDS Tag	0x0b
Size	>= sizeof(struct gapm_set_adv_data_cmd)
Example	OTP_beacon/tag_data/0b-APP_BLE_ADV_DATA/beacon_101.tds
Description	Bluetooth LE Application Advertising data ● For Payload format refer to Bluetooth LE 5.0 core specification Vol 3 Part C Section 11 ADVERTISING AND SCAN RESPONSE DATA FORMAT ● Also refer to https://www.bluetooth.com/specifications/assigned-numbers/ generic-access-profile/" for AD type values.”
Name	**NVDS_TAG_APP_BLE_SCAN_RESP_DATA**
NVDS Tag	0x0c
Size	>= sizeof(struct gapm_set_adv_data_cmd)
Example	OTP_beacon/tag_data/0c-APP_BLE_SCAN_RESP_DATA/beacon_101.tds
Description	Bluetooth LE Application Scan response data ● For Payload format refer to Bluetooth LE 5.0 core specification Vol 3 Part C Section 11 ADVERTISING AND SCAN RESPONSE DATA FORMAT ● Also refer to https://www.bluetooth.com/specifications/assigned-numbers/ generic-access-profile/" for AD type values.”
Name	**PARAM_ID_EXT_WAKEUP_TIME**
NVDS Tag	0x0d
Size	2 bytes

CONFIDENTIAL

Default	2000 (2 ms)
Description	External wake-up time Time for stabilization of the high frequency oscillator following an external wake-up request (in µs).
Name	**PARAM_ID_OSC_WAKEUP_TIME**
NVDS Tag	0x0e
Size	2 bytes
Default	2000 (2 ms)
Description	Oscillator wake-up time Time for stabilization of the high frequency oscillator following a sleep-timer expiry (in µs).
Name	**PARAM_ID_RM_WAKEUP_TIME**
NVDS Tag	0x0f
Size	2 bytes
Default	625 (µs)
Description	Radio wake-up time Time to wake-up Radio Module (in µs).
Name	**PARAM_ID_SLEEP_ENABLE**
NVDS Tag	0x11
Size	1 byte
Default	0 (sleep disabled)
Description	Enable sleep mode 0x00: Sleep disabled 0x01: Deep sleep enabled 0x02: Retain all but DO NOT drop to retention voltage 0x03: Retain all and drop to retention voltage 0x04: Hibernate 0x05: SoC off

CONFIDENTIAL

	0x80: Test mask (roll through modes)
Name	**PARAM_ID_EXT_WAKEUP_ENABLE**
NVDS Tag	0x12
Size	1 byte
Example	OTP_beacon/tag_data/12-EXT_WAKEUP_ENABLE/enable.tds
Description	Enable External Wakeup 0x00: Timer and software wakeup only 0x01: External (from PSEQ) wakeup enabled with limited timeout 0x02: External wakeup enabled with no maximum sleep duration
Name	**PARAM_ID_ACTIVITY_MOVE_CONFIG**
NVDS Tag	0x15
Size	1 byte
Description	Activity Move Configuration Enables/disables activity move for Bluetooth LE connections.
Name	**PARAM_ID_SCAN_EXT_ADV**
NVDS Tag	0x16
Size	1 byte
Default	1 (enabled)
Description	Extended advertisement scanning Enable/disable scanning for extended advertising PDUs.
Name	**PARAM_ID_SCHED_SCAN_DUR**
NVDS Tag	0x17
Size	2 bytes
Default	22500 (18 µs)
Description	Scheduler scan duration

CONFIDENTIAL

	Duration of the schedule reservation for long activities such as scan, inquiry, page, HDC advertising.
Name	**NVDS_TAG_APP_WURX_BLE_ACT_STRT_CMD**
NVDS Tag	0x20
Size	sizeof(struct gapm_adv_param)
Example	OTP_beacon/tag_data/05-APP_BLE_ACT_STRT_CMD/30_adv.tds
Description	Wake-up Rx Bluetooth LE Application Activity Start CMD
Name	**NVDS_TAG_APP_WURX_BLE_ACT_CRT_CMD**
NVDS Tag	0x21
Size	sizeof(struct gapm_activity_create_adv_cmd)
Example	OTP_beacon/tag_data/06-APP_BLE_ACT_CRT_CMD/ref_legacy_nc_s_1s.tds
Description	Wake-up Rx Bluetooth LE Application Activity Create CMD
Name	**NVDS_TAG_APP_WURX_BLE_ADV_DATA**
NVDS Tag	0x22
Size	>= sizeof(struct gapm_set_adv_data_cmd)
Example	OTP_beacon/tag_data/0b-APP_BLE_ADV_DATA/beacon_101.tds
Description	Wake-up Rx Bluetooth LE Application Advertising data ● For Payload format refer to Bluetooth LE 5.0 core specification Vol 3 Part C Section 11 ADVERTISING AND SCAN RESPONSE DATA FORMAT ● Also refer to https://www.bluetooth.com/specifications/assigned-numbers/ generic-access-profile/" for AD type values.”
Name	**NVDS_TAG_APP_WURX_BLE_SCAN_RESP_DATA**
NVDS Tag	0x23
Size	>= sizeof(struct gapm_set_adv_data_cmd)

CONFIDENTIAL

Example	OTP_beacon/tag_data/0c-APP_BLE_SCAN_RESP_DATA/beacon_101.tds
Description	Wake-up Rx Bluetooth LE Application Scan response data ● For Payload format refer to Bluetooth LE 5.0 core specification Vol 3 Part C Section 11 ADVERTISING AND SCAN RESPONSE DATA FORMAT ● Also refer to https://www.bluetooth.com/specifications/assigned-numbers/ generic-access-profile/" for AD type values.”
Name	**PARAM_ID_SLEEP_ADJ**
NVDS Tag	0x2b
Size	4 bytes
Default	10 for FlashROM applications, 0 for all others
Description	Sleep duration adjustment (in half slots which are 312.5 µs). Positive values increase leeway before the next event by reducing the requested sleep time. Negative values can be used to optimize power, but they can also break the sleep algorithm by causing the next event to be missed.
Name	**PARAM_ID_SLEEP_ALGO_DUR**
NVDS Tag	0x2e
Size	2 bytes
Default	500 (250 µs)
Description	Sleep algorithm duration Duration of sleep and wake-up algorithm (depends on CPU speed) expressed in half µs.
Name	**PARAM_ID_DIAG_BLE_HW**
NVDS Tag	0x31
Size	4 bytes
Description	Diagnostic port configuration
Name	**PARAM_ID_DIAG_SW**

CONFIDENTIAL

NVDS Tag	0x32
Size	4 bytes
Description	Diagnostic port configuration
Name	**NVDS_TAG_RECENT_BOND**
NVDS Tag	0x36
Size	2 bytes
Description	Contains the slot index of the last connected keyboard
Name	**NVDS_TAG_MOUSE_SAMPLE_RATE**
NVDS Tag	0x38
Size	1 byte
Description	Mouse Sample Rate Reports per second.
Name	**NVDS_TAG_PERIPH_BONDED**
NVDS Tag	0x39
Size	1 byte
Description	Peripheral Bonded
Name	**NVDS_TAG_MOUSE_NTF_CFG**
NVDS Tag	0x3a
Size	2 bytes
Description	Mouse NTF Configuration
Name	**NVDS_TAG_MOUSE_TIMEOUT**
NVDS Tag	0x3b
Size	2 bytes

CONFIDENTIAL

Default	2000 (20 seconds)
Description	Mouse Timeout value Duration before connection update procedure if no report received (mouse is silent).
Name	**NVDS_TAG_PEER_BD_ADDRESS**
NVDS Tag	0x3c
Size	7 bytes
Description	Peer Device BD Address
Name	**NVDS_TAG_MOUSE_ENERGY_SAFE**
NVDS Tag	0x3d
Size	2 bytes
Default	6000 (60 seconds)
Description	Mouse Energy Safe Duration before disconnection if no report is received after connection update.
Name	**NVDS_TAG_LTK**
NVDS Tag	0x3e
Size	sizeof(struct gapc_ltk) == 28
Description	EDIV (2 bytes), RAND NB (8 bytes), LTK (16 bytes), Key Size (1 byte)
Name	**NVDS_TAG_PERIPH_BONDED_2**
NVDS Tag	0x41
Size	1 byte
Description	Bonded Slot Number 2
Name	**NVDS_TAG_PERIPH_BONDED_3**
NVDS Tag	0x42

CONFIDENTIAL

Size	1 byte
Description	Bonded Slot Number 3
Name	**NVDS_TAG_PEER_BD_ADDRESS_2**
NVDS Tag	0x43
Size	7 bytes
Description	Peer Device BD Address Slot Number 2
Name	**NVDS_TAG_PEER_BD_ADDRESS_3**
NVDS Tag	0x44
Size	7 bytes
Description	Peer Device BD Address Slot Number 3
Name	**PARAM_ID_LE_PRIVATE_KEY_P256**
NVDS Tag	0x80
Size	32 bytes
Description	SC Private Key (Low Energy)
Name	**PARAM_ID_LE_PUBLIC_KEY_P256**
NVDS Tag	0x81
Size	64 bytes
Description	SC Public Key (Low Energy)
Name	**PARAM_ID_LE_DBG_FIXED_P256_KEY**
NVDS Tag	0x82
Size	1 byte
Description	SC Debug: Used Fixed Private Key from NVDS (Low Energy)
Name	**PARAM_ID_LE_CODED_PHY_500**

CONFIDENTIAL

NVDS Tag	0x85
Size	1 byte
Default	0 (false = 125 Kbps)
Description	LE Coded PHY 500 Kbps selection
Name	**NVDS_TAG_LOC_IRK**
NVDS Tag	0xa0
Size	16 bytes
Description	Local device Identity resolving key
Name	**NVDS_TAG_PEER_IRK**
NVDS Tag	0xa1
Size	sizeof(struct gapc_irk)
Description	Peer device Resolving identity key (+identity address)
Name	**NVDS_TAG_PEER_IRK_2**
NVDS Tag	0xa2
Size	sizeof(struct gapc_irk)
Description	Peer device Resolving identity key Slot Number 2(+identity address)
Name	**NVDS_TAG_ADV_DURATION**
NVDS Tag	0xa3
Size	2 bytes
Description	Beacon interval duration in 10 ms unit
Name	**NVDS_TAG_SCAN_SETTING**
NVDS Tag	0xa4
Size	sizeof(struct app_scan_setting)

CONFIDENTIAL

Description	Scan parameters used by BLE_adv_scan app
Name	**NVDS_TAG_WURX_PROFILE**
NVDS Tag	0xa5
Size	sizeof(struct app_wurx_profile)
Description	Wakeup Rx profile information used by BLE_scan_adv app
Name	**NVDS_TAG_PEER_IRK_3**
NVDS Tag	0xa6
Size	sizeof(struct gapc_irk)
Description	Peer device Resolving identity key Slot Number 3(+identity address)
Name	**NVDS_TAG_BRIDGE_SERVICE**
NVDS Tag	0xa7
Size	NVDS_LEN_BRIDGE_NVDS (18 bytes)
Description	The bridge profile characteristic TRX uuid
Name	**NVDS_TAG_CHAR_TRX**
NVDS Tag	0xa8
Size	NVDS_LEN_BRIDGE_NVDS (18 bytes)
Description	The bridge profile characteristic TRX uuid
Name	**NVDS_TAG_CHAR_TRX2**
NVDS Tag	0xa9
Size	NVDS_LEN_BRIDGE_NVDS (18 bytes)
Description	The bridge profile characteristic TRX2 uuid
Name	**NVDS_TAG_PEER_LTK**
NVDS Tag	0xab

CONFIDENTIAL

Size	sizeof(struct gapc_ltk) == 28 bytes
Description	Peer device long term key. EDIV (2bytes), RAND NB (8bytes), LTK (16 bytes), Key Size (1 byte)
Name	**NVDS_TAG_PEER_LTK_2**
NVDS Tag	0xac
Size	sizeof(struct gapc_ltk) == 28 bytes
Description	Peer device long term key slot number 2. EDIV (2bytes), RAND NB (8bytes), LTK (16 bytes), Key Size (1 byte)
Name	**NVDS_TAG_PEER_LTK_3**
NVDS Tag	0xad
Size	sizeof(struct gapc_ltk) == 28 bytes
Description	Peer device long term key slot number 3. EDIV (2bytes), RAND NB (8bytes), LTK (16 bytes), Key Size (1 byte)
Name	**SYDNEY_TAG_RIF_CAL**
NVDS Tag	0xb0
Size	multiple of 4 bytes
Description	Manufacturing calibration data for CMSDK_RIF registers
Name	**SYDNEY_TAG_MDM_CAL**
NVDS Tag	0xb1
Size	multiple of 4 bytes
Description	Manufacturing calibration data for CMSDK_MDM registers
Name	**SYDNEY_TAG_BLE_RADIO_OVR**
NVDS Tag	0xb2
Size	sizeof(struct BLE_radio_ovr_s)

CONFIDENTIAL

Description	Override Bluetooth LE LC configuration
Name	**SYDNEY_TAG_FLASH_POWERUP_DELAY**
NVDS Tag	0xb3
Size	2 bytes
Description	Delay (in 32 KHz cycles) for first flash access after power on
Name	**SYDNEY_TAG_PMU_WURX**
NVDS Tag	0xb4
Size	multiple of 4, <= sizeof(struct PMU_wurx_s)
Description	PMU configuration for wake-up receiver
Name	**SYDNEY_TAG_WATCHDOG**
NVDS Tag	0xb5
Size	4 bytes
Default	Watchdog disabled
Description	Timeout (in 16 MHz cycles)
Name	**SYDNEY_TAG_USER_APP**
NVDS Tag	0xb6
Size	variable up to OTP storage max
Description	OTP user application
Name	**SYDNEY_TAG_PMU_WKUP**
NVDS Tag	0xb7
Size	4 bytes
Description	Initialization value for PMU_TOP__PMU1_REG For PIN and LPCOMP settings in PMU_TOP__PMU1_REG, refer to PMU_TOP section of Register Specification document.

CONFIDENTIAL

	This tag is applicable for OTP based applications. For Flash applications, refer to pmu_socoff_wakeup_* APIs in platform/atm2/ATM2xE3c/driver/pmu/pmu.h.
Name	**SYDNEY_TAG_ATE**
NVDS Tag	0xb8
Size	variable
Description	Process data written during manufacturing
Name	**SYDNEY_TAG_HIBERNATE_ADJ**
NVDS Tag	0xb9
Size	4 bytes (signed integer)
Default	0x40
Description	Adjustment (in 32 KHz cycles) Subtracted from Bluetooth LE sleep duration to obtain PSEQ hibernate duration.
Name	**SYDNEY_TAG_RETAIN_ADJ**
NVDS Tag	0xba
Size	4 bytes (signed integer)
Default	0x20
Description	Adjustment (in 32 KHz cycles) Subtracted from Bluetooth LE sleep duration to obtain PSEQ retain duration.
Name	**SYDNEY_TAG_SLEEP_MIN**
NVDS Tag	0xbb
Size	4 bytes (signed integer)
Default	0x20
Description	Minimum sleep duration (in 32 KHz cycles) Minimum PSEQ duration to engage power saving mode. Smaller durations will only utilize Bluetooth LE sleep.

CONFIDENTIAL

Name	SYDNEY_TAG_MISC_CAL
NVDS Tag	0xbc
Size	<= sizeof(struct misc_cal_s)
Description	Manufacturing calibration data
Name	**SYDNEY_TAG_CUST_CFG**
NVDS Tag	0xbd
Size	<= sizeof(struct cust_cfg_s)
Description	Customer-provided design-specific calibration data
Name	**SYDNEY_TAB_RADIO_CAL**
NVDS Tag	0xbe
Size	<= sizeof(struct radio_cal_s)
Description	Manufacturing calibration data for RADIO registers
Name	**SYDNEY_TAG_LED_ENABLE**
NVDS Tag	0xbf
Size	1 byte
Description	Mask to enable status outputs for internal ROM operations. Bit 0, P3_GPIO_3_: Deep sleep Bit 1, P4_GPIO_4_: LED0 ? Hibernation : CPU WFI Bit 2, P5_GPIO_5_: Bluetooth LE sleeping Bit 3, P6_GPIO_6_: Retention Bit 4, P7_GPIO_7_: ECC computation Bit 5, P8_GPIO_8_: Active tick Bit 6, P9_GPIO_9_: Bluetooth LE waking up
Name	**SYDNEY_TAG_PMU_W**
NVDS Tag	0xfc
Size	6, 4, or 3 bytes

CONFIDENTIAL

Description	Arbitrary 32, 16, 8-bit PMU register write
Name	**SYDNEY_TAG_MEM_W**
NVDS Tag	0xfd
Size	8, 6, or 5 bytes
Description	Arbitrary 32, 16, 8-bit register write
Name	**SYDNEY_TAG_MEM_RMW**
NVDS Tag	0xfe
Size	12, 8, or 6 bytes
Description	Arbitrary 32, 16, 8-bit register read-modify-write
Name	**NVDS_TAG_APP Tags**
NVDS Tag	0xd0 to 0xef
Description	Reserved for application usage. Application can define and use tags within this range.

CONFIDENTIAL

8 ROM Buffers and Capacities

This section describes various buffers allocated by the ROM and their capacities.

8.1 Bluetooth LE heaps

(gdb) p sizeof(rwip_heap_env)
$1 = 4768
(gdb) p sizeof(rwip_heap_db)
$2 = 3084
(gdb) p sizeof(rwip_heap_msg)
$3 = 17152
(gdb) p sizeof(rwip_heap_non_ret)
$4 = 668

8.2 Bluetooth LE Exchange Memory

EM_ET_OFFSET = 0
EM_ET_END = 256
EM_FT_END = 296
EM_RF_SW_SPI_END = 302
EM_RF_HW_SPI_END = 302
EM_COMMON_END = 302
EM_ENC_END = 334
EM_BLE_CS_OFFSET = 334
EM_BLE_CS_END = 1054
EM_BLE_WL_OFFSET = 1054
EM_BLE_WL_END = 1118
EM_BLE_RAL_OFFSET = 1118
EM_BLE_RAL_END = 1274
EM_BLE_RX_DESC_OFFSET = 1274
EM_BLE_RX_DESC_END = 1394
EM_BLE_TX_DESC_OFFSET = 1394
EM_BLE_TX_DESC_END = 1982
EM_BLE_LLCPTXBUF_END = 2270
EM_BLE_ADVEXTHDRTXBUF_END = 2648
EM_BLE_ADVDATATXBUF_END = 10268
EM_BLE_AUXCONNECTREQTXBUF_END = 10370
EM_BLE_DATARXBUF_END = 12674
EM_BLE_ACLTXBUF_END = 14722
EM_BLE_ISO_BUFFER_END = 14722
EM_BLE_END = 14722

CONFIDENTIAL

Appendix A - Known Issues

This section documents known issues in the ROM and provides workarounds, when possible.

Issue 9	Periodic scans with duplicate filtering is unsupported
Description	The GAP interface specification lists GAPM_DUP_FILT_EN_PERIOD (0x02) as a valid configuration to reset the filter cache at the beginning of each scan period. However, the current ROM does not support this feature.
Workaround	Periodic scans with duplicate filtering can be emulated using normal scans (with duplicate filtering enabled) scheduled by a software timer.
Issue 10	**Two timers about ping operation are left running after disconnect**
Description	When a encrypted link start, the LLC_AUTH_PAYL_REAL_TO and LLC_AUTH_PAYL_NEARLY_TO timers will be set to keep pinging the peer. After the link was disconnected, these two timers were left. The impact of these timers running is delaying hibernation up to 30s.
Workaround	The ROM_errata_10.c code will patch the message handler for LLC_STOPPED_IND by stopping these two timers before running the original handler. (llc_stopped_ind_handler)
Issue 11	**Enter slave latency when the last RX attempt is failed except sync miss**
Description	When the slave device wakes up from the slave latency anchor, it will only try to receive packets on the next connection event only if the packet on this anchor has a sync error. It means that the slave will still enter slave latency when the packet was sync-passed but CRC failed. It also happens when there is a NESN error, which means the peer side didn't receive the last Tx packet. This will increase the risk of disconnection.
Workaround	The ROM_errata_14.c code will add a patched function into the lld connection schedule vector to replace the old lld_con_sched function. The new patched function will make sure the latest Rx attempt is successful and then enter slave latency.

CONFIDENTIAL



CONFIDENTIAL

ATMOSIC TECHNOLOGIES - DISCLAIMER

This product document is intended to be a general informational aid and not a substitute for any literature or labeling accompanying your purchase of the Atmosic product. Atmosic reserves the right to amend its product literature at any time without notice and for any reason, including to improve product design or function. While Atmosic strives to make its documents accurate and current, Atmosic makes no warranty or representation that the information contained in this document is completely accurate, and Atmosic hereby disclaims (i) any and all liability for any errors or inaccuracies contained in any document or in any other product literature and any damages or lost profits resulting therefrom; (ii) any and all liability and responsibility for any action you take or fail to take based on the information contained in this document; and (iii) any and all implied warranties which may attach to this document, including warranties of fitness for particular purpose, non-infringement and merchantability. Consequently, you assume all risk in your use of this document, the Atmosic product, and in any action you take or fail to take based upon the information in this document. Any statements in this document in regard to the suitability of an Atmosic product for certain types of applications are based on Atmosic's general knowledge of typical requirements in generic applications and are not binding statements about the suitability of Atmosic products for any particular application. It is your responsibility as the customer to validate that a particular Atmosic product is suitable for use in a particular application. All content in this document is proprietary, copyrighted, and owned or licensed by Atmosic, and any unauthorized use of content or trademarks contained herein is strictly prohibited.

Copyright ©2021 by Atmosic Technologies. All rights reserved.

www.atmosic.com